Projeto de intervenção — Situação-Problema 3

Nome: _____

RGM: _____

Qual situação-problema você escolheu para criar o seu projeto de intervenção?

- [ ] Situação-problema 1
- [ ] Situação-problema 2
- [X] Situação-problema 3

A Situação-Problema escolhida:

Escolhi a Situação-Problema 3, Refatoração e Orientação a Objetos, apresentada no material Projetos Computacionais: da Teoria à Prática, de Artur Marques, nas páginas 39–40. O cenário descreve um catálogo de livros e filmes em que cadastro, listagem, consulta de detalhes, edição, exclusão e busca estão concentrados em uma Servlet, junto com as regras de negócio e o acesso ao banco de dados.

O problema central é a dificuldade de manutenção causada por essa concentração de responsabilidades. Uma alteração em uma regra de validação ou em uma consulta SQL exige modificar o mesmo componente, e operações semelhantes acabam repetidas. Delimitei a intervenção à reorganização desse catálogo, preservando suas funcionalidades e separando apresentação, controle das requisições, regras de negócio e persistência.

Desenvolvi a solução com o nome Acervo, contemplando livros, filmes e séries. O trabalho foi realizado em ambiente local, com análise do cenário, modelagem, implementação e verificação do sistema. O foco da análise foi a organização do código e o funcionamento das operações do catálogo.

Objetivo:

Reorganizar o catálogo Acervo por meio da orientação a objetos e de uma arquitetura em camadas, para facilitar a manutenção, reduzir a repetição de código e permitir testes das regras de negócio, mantendo o cadastro, a listagem, a consulta de detalhes, a edição, a exclusão, a busca e o armazenamento dos dados.

Para alcançar esse objetivo, defini as seguintes ações: representar as obras na entidade ItemMidia; concentrar o acesso ao banco no DAO; centralizar as validações no serviço; manter a Servlet responsável pelo fluxo HTTP; e utilizar páginas JSP para apresentar as informações. Também estabeleci como critérios de conclusão o funcionamento do CRUD e da busca, a conservação dos dados em arquivo e a execução dos testes de serviço, persistência e integração web.

Revisão de Conceito:

A fundamentação do trabalho parte dos conceitos de Java e Programação Orientada a Objetos abordados no material de Marques (s.d.). Classes reúnem estado e comportamento, enquanto objetos representam as instâncias utilizadas pela aplicação. Apliquei esses conceitos na entidade ItemMidia, que representa os dados de uma obra, e nas classes responsáveis pelas operações do catálogo.

O encapsulamento protege o estado interno e organiza o acesso aos dados. Em ItemMidia, utilizei atributos privados e métodos de leitura. A abstração permite definir o que uma operação deve oferecer sem expor seus detalhes internos; no projeto, a interface ItemMidiaDao estabelece o contrato de persistência. O polimorfismo permite utilizar implementações diferentes desse contrato: JdbcItemMidiaDao acessa o banco, enquanto uma implementação em memória atende aos testes de serviço. A herança aparece em ItemMidiaServlet, que estende HttpServlet e especializa o atendimento das requisições.

O padrão DAO concentra as operações de acesso aos dados, evitando que consultas SQL se espalhem pelo controlador. A separação em camadas favorece a coesão, pois cada classe reúne tarefas relacionadas, e reduz o acoplamento, pois o serviço depende de um contrato de persistência. A organização MVC distribui o modelo entre a entidade e as regras de negócio, a visualização nas JSPs e o controle do fluxo HTTP na Servlet.

Na implementação, relacionei esses conceitos às tecnologias utilizadas. Servlets e JSPs são executadas pelo Tomcat; Jakarta Tags/JSTL permite percorrer coleções e apresentar textos nas páginas; JDBC realiza as operações no banco relacional H2. Utilizei consultas parametrizadas com PreparedStatement e fechamento de recursos com try-with-resources. As documentações de Tomcat, Jakarta Tags e H2, relacionadas nas referências, complementam a aplicação prática dos conceitos.

Metodologia:

Adotei uma abordagem prática de desenvolvimento iterativo, orientada pela Aprendizagem Baseada em Projetos. Parti da análise da situação-problema, defini os requisitos, organizei a modelagem e implementei a solução por funcionalidades. A cada etapa, conferi o comportamento esperado antes de avançar para a integração das camadas.

Usei o cenário de Servlet concentrando responsabilidades como referência para identificar o que precisava ser separado. A partir dele, construí a arquitetura do Acervo e comparei a distribuição das responsabilidades com a organização descrita no enunciado. A análise se concentrou no código, nos diagramas e nos resultados dos testes da aplicação desenvolvida.

Organizei a verificação em três níveis: testes de serviço com DAO em memória, testes de persistência com JDBC e H2 e testes HTTP com Tomcat e renderização das JSPs. Os casos abrangem operações válidas, campos incorretos, itens inexistentes, persistência em arquivo, entradas com aspas, tentativas de injeção SQL, escape de HTML e proteção das operações POST por token CSRF. Registrei as evidências nos relatórios de testes e nas capturas de tela.

Tempo:

Realizei o trabalho ao longo de dez dias, distribuindo as atividades entre análise, modelagem, implementação, testes e documentação. Nos primeiros dias, concentrei a atenção na compreensão do problema e na definição da solução. Depois, desenvolvi e integrei as camadas do catálogo. Na etapa final, revisei os testes, organizei os documentos e preparei a apresentação.

Período	Atividades
Dias 1 e 2	Leitura do material, análise da situação-problema e definição dos requisitos e objetivos.
Dias 3 e 4	Modelagem dos casos de uso, classes, banco de dados e sequência das operações; organização da arquitetura.
Dias 5 a 7	Implementação das entidades, persistência, serviço, Servlet e telas; integração do CRUD e da busca.
Dias 8 e 9	Testes de serviço, JDBC e HTTP; revisão das validações, mensagens e persistência dos dados.
Dia 10	Revisão do relatório, manual, guia do código, anexos e roteiro de apresentação.
Total	Dez dias.

Procedimento e material utilizado:

• Analisei o enunciado e identifiquei a mistura de responsabilidades na Servlet como o problema principal.

• Defini as operações do catálogo, os campos das obras e as regras de validação, mantendo o foco em um catálogo pessoal acadêmico.

• Organizei os diagramas de casos de uso, classes, entidade-relacionamento e sequência de cadastro, além do esquema SQL da tabela item_midia.

• Preparei o ambiente com JDK 17, Maven, editor de código e navegador, utilizando Tomcat para executar a aplicação web.

• Implementei ItemMidia e TipoMidia, a criação das conexões e a inicialização do banco H2 em arquivo.

• Defini a interface ItemMidiaDao e implementei JdbcItemMidiaDao para cadastro, consulta, edição, exclusão e busca com SQL parametrizado.

• Centralizei em ItemMidiaService as validações de campos obrigatórios, tamanho dos textos, tipo de mídia e ano de lançamento.

• Organizei ItemMidiaServlet para receber as requisições, chamar o serviço e selecionar as páginas JSP ou o redirecionamento adequado.

• Construí as telas de listagem, cadastro, edição, detalhes e erro com JSP/JSTL, HTML, CSS e JavaScript, incluindo mensagens de validação e confirmação de exclusão.

• Verifiquei as operações e os cenários de falha por meio dos testes de serviço, banco e HTTP, incluindo a conservação dos registros após a reabertura do banco.

• Reuni o relatório técnico, o manual, o guia do código, as capturas de tela, os diagramas e os registros de testes nos arquivos de entrega.

Utilizei o material teórico da disciplina, computador, JDK 17, Maven, Tomcat 10.1, H2, JDBC, Servlets, JSP/JSTL, JUnit 5, editor de código e navegador. Para preparar as versões em Word e PDF, utilizei Python e bibliotecas de geração de documentos.

Resultado e discussão:

Ao final dos dez dias, concluí uma aplicação web com cadastro, listagem, consulta de detalhes, edição, exclusão e busca de obras. O banco H2 conserva os registros em arquivo. A implementação distribui as responsabilidades entre entidade, DAO, serviço, Servlet e JSPs, permitindo localizar as regras de validação e as consultas SQL em componentes próprios.

Avaliei a proposta a partir dos objetivos definidos e das evidências disponíveis. Os registros XML da entrega apresentam 39 testes aprovados: 23 de serviço, 8 de persistência JDBC e 8 de integração HTTP, com zero falhas, erros ou testes ignorados. O detalhamento está em Verificacao_Testes, acompanhado dos arquivos XML em entrega/evidencias_testes.

Critério de conclusão	Resultado observado	Evidência
Preservar as funcionalidades	Cadastro, listagem, detalhes, edição, exclusão e busca implementados.	Testes de CRUD e integração HTTP; capturas de tela.
Separar as responsabilidades	SQL no DAO, validações no serviço e fluxo HTTP na Servlet.	Código-fonte, diagrama de classes e relatório técnico.
Reutilizar o acesso aos dados	Mapeamento de registros e preenchimento de parâmetros compartilhados entre operações.	Métodos mapear e preencher de JdbcItemMidiaDao.
Testar regras de forma isolada	Serviço verificado com DAO em memória.	23 testes de ItemMidiaServiceTest.
Conservar os registros	Dados recuperados após fechamento e reabertura do banco em arquivo.	Teste dadosSobrevivemAoFechamentoEReabertura.

A principal melhoria está na organização do código e na possibilidade de verificar cada responsabilidade separadamente. Ao alterar uma validação, posso trabalhar no serviço; ao modificar uma consulta, no DAO. A interface de persistência permite substituir a implementação usada nos testes sem modificar as regras de negócio. Essa estrutura atende ao problema de manutenção descrito no enunciado.

O projeto também permitiu relacionar os conceitos de POO ao fluxo completo de uma aplicação web, desde o preenchimento do formulário até a gravação no banco. Como continuidade, considero a inclusão de paginação, autenticação, histórico de alterações e controle de edição

simultânea. A avaliação realizada se concentra no funcionamento e na estrutura do sistema, com apoio dos testes automatizados e das telas registradas.

Materiais e anexos:

- Relatorio_Tecnico: contexto, requisitos, arquitetura, modelagem e discussão dos resultados.
- Guia_Detalhado_Codigo: responsabilidades dos arquivos, classes, métodos e fluxos.
- Manual_Usuario: instalação e operação do catálogo.
- Verificacao_Testes e entrega/evidencias_testes: resultados e registros XML dos testes.
- docs/diagramas: diagramas de casos de uso, classes, dados e sequência.
- docs/telas: capturas da listagem, cadastro, detalhes, busca, edição e visualização em tela pequena.
- src: código-fonte e testes; entrega/catalogo.war: aplicação empacotada.
- Roteiro_Apresentacao: sequência de exposição do problema, solução e demonstração.

Referências:

MARQUES, Artur. Projetos Computacionais: da Teoria à Prática. Material teórico da disciplina. [s.d.]. Situação-Problema 3: p. 39–40.

APACHE SOFTWARE FOUNDATION. Apache Tomcat 10.1 Documentation. Disponível em: https://tomcat.apache.org/tomcat-10.1-doc/index.html. Acesso em: 17 out. 2026.

APACHE SOFTWARE FOUNDATION. Apache Maven: Download and Documentation. Disponível em: https://maven.apache.org/download.cgi. Acesso em: 17 out. 2026.

ECLIPSE FOUNDATION. Jakarta Standard Tag Library 3.0. Disponível em: https://jakarta.ee/specifications/tags/3.0/. Acesso em: 17 out. 2026.

H2 DATABASE ENGINE. Features. Disponível em: https://h2database.com/html/features.html. Acesso em: 17 out. 2026.

JUNIT TEAM. JUnit 5.12.2 Release Notes. Disponível em: https://docs.junit.org/5.12.2/release-notes/. Acesso em: 17 out. 2026.